

NALMOBench: Towards Benchmarking Natural Language Interfaces for Moving Objects Databases

Xieyang Wang

Nanjing University of Aeronautics and Astronautics,
China

xieyang@nuaa.edu.cn

Mengyi Liu*

Nanjing University of Aeronautics and Astronautics,
China

liumengyi@nuaa.edu.cn

Weijia Yi

Nanjing University of Aeronautics and Astronautics,
China

wjyi_x@nuaa.edu.cn

Chenchen Zong

Nanjing University of Aeronautics and Astronautics,
China

chencz@nuaa.edu.cn

ABSTRACT

Existing natural language interfaces for databases (NLDBs) benchmarks target relational databases and lack unified quality metrics, support for complex spatio-temporal multi-dimensional queries, and real-world validation of translation accuracy and efficiency. To fill this gap, we propose a benchmark for natural language query (NLQ) translation in moving objects databases (MODs). The benchmark covers five commonly encountered moving objects queries and two real-world scenario queries. Such NLQs are generated to construct the corpus with the usage of large language models (LLMs). Robust NLQs are extracted from the corpus and annotated with the assistance of automatic processes and domain experts. The annotation process incorporates formal verification to ensure query quality. We propose a two-phase query transformation baseline method, including natural language understanding and structured language generation for comprehending the intent of NLQs and generating optimized executable languages, respectively. Experimental results indicate that existing methods struggle to be directly applied in designing NLDBs for MODs, failing to effectively handle complex MOD queries and lack precise control over query efficiency. This highlights significant research potential in NLDB for MODs. The method proposed in this paper achieves average translatability, translation precision, and translation efficiency score of 68.96%, 60.07%, and 68.37, respectively.

VLDB Workshop Reference Format:

Xieyang Wang, Weijia Yi, Mengyi Liu, and Chenchen Zong. NALMOBench: Towards Benchmarking Natural Language Interfaces for Moving Objects Databases. VLDB 2025 Workshop: LLM+Spatial.

1 INTRODUCTION

In recent years, the development of spatio-temporal databases has been driven by an ever-increasing demand for sophisticated management of location-aware data. Natural language, being the most

familiar means of communication, has become the preferred choice for non-experts to use databases. Indeed, advances in LLM-based text-to-SQL techniques have catalyzed the development of natural language interfaces for domain-specific databases. However, standard SQL falls short of capturing the rich spatial and temporal operators required for deep spatio-temporal semantic reasoning, whereas native spatio-temporal engines expose an extensive set of executable languages that can express complex queries [12]. To bridge this gap, researchers have begun exploring direct translation from natural language into executable languages (text-to-EXE). Yet this emerging field lacks a unified benchmark and public datasets. Referring to the maturity of text-to-SQL benchmarks, we need to discuss the following three questions.

Database: database about simulated train movement trajectories in Berlin consisting of 51,544 moving object points.

Example NLQ: Find the 5 continuous nearest neighbors to the train 100 between 6:00 and 19:00 o'clock.

GPT-4o predicted SQL:

```
SELECT A.Id
FROM Trains AS A, Trains AS B
WHERE B.Id = 100
AND StartPoint(Intersection(A.Trip, B.Trip)) BETWEEN mpoint('2020-11-20
06:00:00') AND mpoint('2020-11-20 19:00:00')
ORDER BY Distance(StartPoint(A.Trip), StartPoint(B.Trip))
LIMIT 5;
```

Executable language of SECONDO:

```
let t100 = Trains feed filter[.Id = 100] extract[Trip];
query UTOOrdered_rtree UTOOrdered feed filter [(deftime(UTrip) intersects [const
periods value (('2020-11-20-06:00' '2020-11-20-19:00' TRUE TRUE)])])
nearest_neighbor[UTrip, t100, 5] consume;
```

Figure 1: An example of GPT-4o predicted SQL and the corresponding executable language of SECONDO. White parts are entities. Bold and underlined parts are operators used for explaining the structured semantics expression of nearest.

Q1: Why do we need text-to-EXE compared to text-to-SQL for moving objects databases? While significant advances have been made, most achievements focus on relational databases, such as the rule-based method NaLIR [6], neural network-based method IRNet [3], and LLM-based methods like FinSQL [16]. Relational databases using SQL for querying may encounter challenges when

*Corresponding author

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, ISSN 2150-8097.

NALMOBench: 面向移动对象数据库自然语言接口的基准测试

王谢阳 南京航空航天大学
，中国 xieyang@nuaa.edu.cn

魏嘉 南京航空航天大学
，中国 wji_x@nuaa.edu.cn

刘萌怡* 中国南京航空航天大学
大学 liumengyi@nuaa.edu.cn

陈晨 宗 南京航空航天大学
，中国 chencz@nuaa.edu.cn

摘要

现有的数据库自然语言接口（NLIDs）基准主要针对关系数据库，缺乏统一的质量指标、对复杂时空多维查询的支持，以及对翻译准确性和效率的真实世界验证。为填补这一空白，我们提出了一个面向移动对象数据库（MODs）的自然语言查询（NLQ）翻译基准。该基准涵盖五类常见的移动对象查询以及两个真实场景查询。这些NLQ通过使用大型语言模型（LLMs）生成，以构建语料库。从语料库中提取出稳健的NLQ，并在自动化流程和领域专家的协助下进行标注。标注过程中融入形式化验证，以确保查询质量。我们提出了一种两阶段查询转换基线方法，包括自然语言理解和结构化语言生成，用于理解NLQ的意图并生成优化的可执行查询。

VLDB 研讨会参考格式：

王谢杨、易伟佳、刘梦怡、宗晨晨。NALMOBench: 迈向移动对象数据库自然语言接口的基准测试。VLDB 2025 研讨会：LLM+Spatial。

1 介绍

近年来，时空数据库的发展是由对位置感知数据的复杂管理日益增长的需求所推动的。自然语言，作为最

熟悉的交流方式，已成为非专业人士使用数据库的首选方式。实际上，基于大语言模型（LLM）的文本到 SQL 技术的进步促进了面向特定领域数据库的自然语言接口的发展。然而，标准 SQL 无法捕捉用于深层时空语义推理的丰富时空操作符，而原生时空引擎则提供了广泛的可执行语言集合，可以表达复杂的查询 [12]。为弥合这一差距，研究人员开始探索从自然语言直接翻译为可执行语言（text-to-EXE）。然而，这一新兴领域缺乏统一的基准和公共数据集。参考文本到 SQL 基准的成熟度，我们需要讨论以下三个问题。

数据库：关于柏林模拟列车运动轨迹的数据库，包括51,544个移动对象点。

Example NLQ: Find the 5 continuous nearest neighbors to the train 100 between 6:00 and 19:00 o'clock.

GPT-4o predicted SQL:

```
SELECT A.Id
FROM Trains AS A, Trains AS B
WHERE B.Id = 100
AND StartPoint(Intersection(A.Trip, B.Trip)) BETWEEN mpoint('2020-11-20
06:00:00') AND mpoint('2020-11-20 19:00:00')
ORDER BY Distance(StartPoint(A.Trip), StartPoint(B.Trip))
LIMIT 5;
```

Executable language of SECONDO:

```
let t100 = Trains feed filter[.Id = 100] extract[Trip];
query UTOOrdered_rtree UTOOrdered feed filter [(deftime(UTrip) intersects [const
periods value (('2020-11-20-06:00" "2020-11-20-19:00" TRUE TRUE))]]]
nearest_neighbor[UTrip, t100, 5] consume;
```

图 1: GPT-4o 预测的 SQL 示例及相应的 SECONDO 可执行语言。白色部分是实体。加粗且带下划线的部分是由于解释最近邻结构语义表达的运算符。

Q1: 相比于文本到SQL，对于移动对象数据库，我们为什么需要文本到EXE？尽管取得了显著进展，但大多数成果集中在关系数据库上，例如基于规则的方法NaLIR [6]、基于神经网络的方法IRNet [3]，以及基于大语言模型的方法如FinSQL [16]。使用SQL进行查询的关系数据库在遇到

*Corresponding author

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment. ISSN 2150-8097.

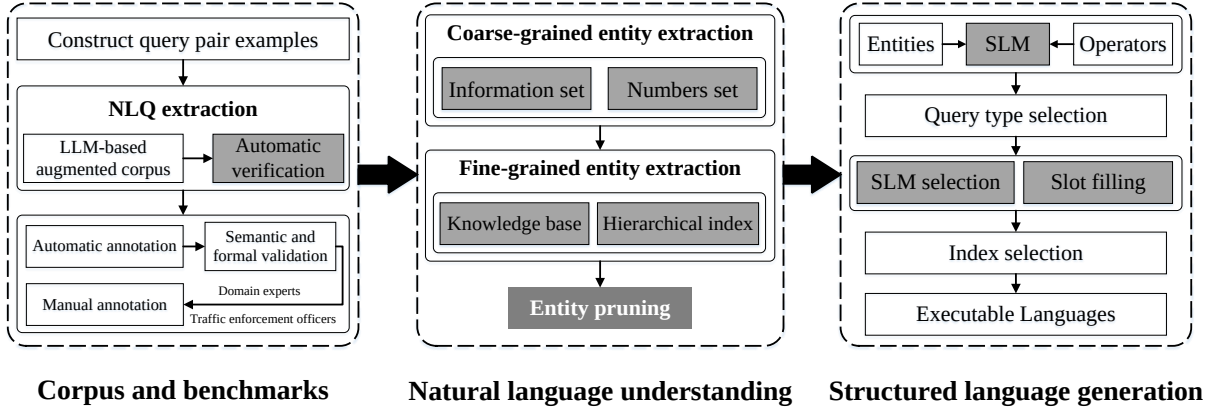


Figure 2: Overview of the benchmark.

handling complex moving objects queries involving multidimensional data. Existing NLDBs often depend on optimizers for query optimization, which can result in reduced control over data and performance. In contrast, executable languages offer the possibility of solving semantic parsing and query translation due to the support for a wide range of spatio-temporal queries and operators, and can directly optimize the query with index usage and operator selection. As illustrated in Figure 1, the SQL generated by GPT-4o necessitates a comprehensive combination of operators and predicates to accurately represent the semantics of a nearest neighbor query. Executable languages can convey the same semantics just using the *nearest_neighbor* operator. Additionally, the executable database query can optionally utilize the 3D R-tree index, represented by *UOrdered_rtree*. Note that subsequent executable languages are exemplified using the executable languages of SECONDO [4], which is a real MOD system.

Q2: Can LLMs dominate the text-to-EXE landscape? As mentioned previously, LLM-based methods now dominate the text-to-SQL landscape, leading us to consider whether LLMs can be effectively used for text-to-EXE [9]. Yet LLMs face two principal obstacles when generating executable languages. On one hand, existing approaches rely on task-specific corpus for executable languages, which often necessitates prompt engineering or costly fine-tuning with newly created datasets. Relying on prompts introduces a trade-off in accuracy of entity linking, operator selection, and index usage. Additionally, the design and creation of specialized corpus for fine-tuning are often cost-prohibitive. On the other hand, even with the usage of prompts or fine-tuning, the accuracy of entity linking in the text-to-SQL domain remains below 75%, let alone for complex spatio-temporal queries. To comprehensively investigate whether existing translation methods and LLMs can offer valuable insights for text-to-EXE mechanisms in MODs, a benchmark is demanded.

Q3: How to evaluate text-to-EXE systems? Mainstream benchmarks primarily focus on text-to-SQL evaluation, failing to address the complex spatio-temporal query requirements of MODs. Early datasets like Geoquery [15] only provide NLQs without corresponding SQL. While WikiSQL [18] offers NLQ and SQL pairs but only for simple queries. Spider [14] and Bird [7] support cross-domain

queries but target fixed datasets, lacking specialized evaluation for complex MOD queries. MOD queries typically represent real-world needs. Current benchmarks exhibit two fundamental limitations: (i) the text-to-SQL focus neglects domain-specific text-to-EXE evaluation and (ii) existing assessments lack real-world moving objects scenario validation, overemphasizing translation accuracy while underrepresenting executable query efficiency. These gaps necessitate specialized benchmarks that capture domain-specific characteristics and semantics, coupled with a comprehensive evaluation framework balancing both accuracy and efficiency.

Challenges. To construct an in-depth benchmark for text-to-EXE functionality within the context of MODs, we should tackle three primary challenges: (i) **The lack of domain knowledge-based benchmark construction.** The complex domain knowledge of MODs and the wide range of extensible operators pose difficulties for the construction and annotation of NLQs. (ii) **The hardness of natural language understanding.** Both spatial and temporal information can be ambiguous. The synonymous, abbreviated and ambiguous entities need to be considered when designing queries. (iii) **The complexity of structured language generation.** Due to the diversity of operators and richness of query types, structured language generation must simultaneously address the composition of entities and operators, and query optimization challenges.

To address these limitations, we present NALMOBench, an LLM-driven benchmark for evaluating text-to-EXE translation in MODs. The complex real-world benchmark assesses the performance of text-to-EXE systems and LLMs. Our proposed LLM-based automated corpus generation method efficiently produces large-scale annotated text-to-EXE datasets with domain generalization capabilities, while simultaneously addressing natural language understanding and structured language generation challenges. We also provide baseline methods for comparison. To the best of our knowledge, NALMOBench represents the first benchmark collaboratively developed by spatio-temporal database experts and traffic law enforcement officers. The benchmark comprehensively supports standard moving objects queries, including time interval queries, range queries, nearest neighbor queries, join queries, and trajectory similarity queries, and real-world traffic enforcement scenarios. Unique among existing benchmarks, NALMOBench jointly optimizes for

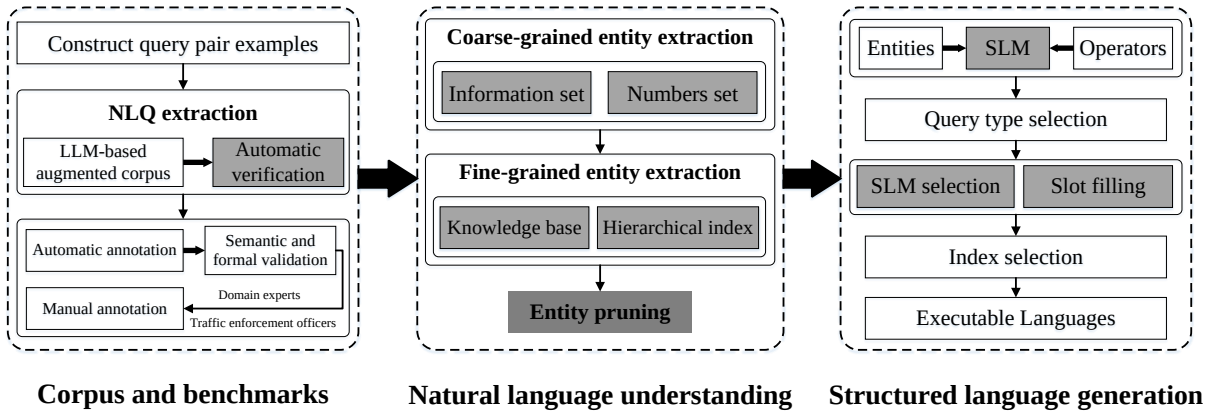


图 2: 基准概览。

处理涉及多维数据的复杂移动对象查询。现有的自然语言接口数据库 (NLIDBs) 通常依赖优化器进行查询优化, 这可能导致对数据和性能的控制能力下降。相比之下, 可执行语言由于支持广泛的时空查询和操作符, 提供了解决语义解析和查询转换的可能性, 并且可以通过索引使用和操作符选择直接优化查询。如图1所示, GPT-4生成的SQL需要综合使用多种操作符和谓词才能准确表示最近邻查询的语义。可执行语言仅使用nearest_neighbor操作符即可表达相同的语义。此外, 可执行数据库查询可以选择性地利用由UTOrdered_rtree表示的3D R树索引。请注意, 后续的可执行语言示例均采用SECONDO [4] 的可执行语言, 该系统是一个真实的多维对象数据库。

Q2: LLM能否主导文本到EXE的领域? 如前所述, 基于LLM的方法现在主导了文本到SQL的领域, 这使我们考虑LLM是否可以有效用于文本到EXE。然而, LLM在生成可执行语言时面临两个主要障碍。一方面, 现有的方法依赖于可执行语言的特定任务语料库, 这通常需要提示工程或使用新创建的数据集进行昂贵的微调。依赖提示会在实体链接、操作符选择和索引使用的准确性之间引入权衡。此外, 为微调设计和创建专门的语料库通常成本高昂。另一方面, 即使使用提示或微调, 文本到SQL领域的实体链接准确率仍低于75%, 更不用说复杂的时空查询了。为了全面研究现有的翻译方法和LLM是否能够为文本到EXE提供有价值的见解

第三问: 如何评估文本到 EXE 系统? 主流基准主要关注文本到 SQL 的评估, 未能解决 MOD 的复杂时空查询需求。早期数据集如 Geoquery [15] 仅提供自然语言查询 (NLQs), 没有对应的 SQL。虽然 WikiSQL [18] 提供了 NLQ 和 SQL 配对, 但仅适用于简单查询。Spider [14] 和 Bird [7] 支持跨领域。

查询但目标固定数据集, 缺乏对复杂MOD查询的专业评估。MOD查询通常代表现实世界的需求。目前的基准存在两个基本局限性: (i) 文本到SQL的关注忽略了特定领域文本到可执行查询的评估, 以及(ii) 现有评估缺乏现实移动对象场景验证, 过度强调翻译准确性而低估了可执行查询的效率。这些差距需要专门的基准来捕捉特定领域的特征和语义, 并配合一个平衡准确性和效率的综合评估框架。

挑战。为了在MOD的背景下构建一个深入的文本到EXE可能的基准, 我们应当应对三个主要挑战: (i) 缺乏基于领域知识的基准构建。MOD的复杂领域知识以及广泛的可扩展操作符给自然语言查询 (NLQs) 的构建和标注带来了困难。(ii) 自然语言理解的难度。空间和时间信息都可能具有歧义。在设计查询时, 需要考虑同义词、缩写和模糊实体。(iii) 结构化语言生成的复杂性。由于操作符的多样性和查询类型的丰富性, 结构化语言生成必须同时解决实体与操作符的组合以及查询优化的挑战。

为了解决这些局限性, 我们提出了NALMOBench, 这是一个基于大型语言模型 (LLM) 的基准, 用于评估MODs中的文本到EXE的转换能力。该复杂的真实世界基准评估文本到EXE系统和LLM的性能。我们提出的基于LLM的自动语料生成方法能够高效地生成大规模带注释的文本到EXE数据集, 并具备领域泛化能力, 同时解决自然语言理解和结构化语言生成的挑战。我们还提供了用于比较的基线方法。据我们所知, NALMOBench是首个由时空数据库专家和交通执法人员共同开发的基准。该基准全面支持标准动态对象查询, 包括时间区间查询、范围查询、最近邻查询、连接查询和轨迹相似性查询, 以及真实世界的交通执法场景。在现有基准中独树一帜的是,

both translation accuracy and execution efficiency. Figure 2 illustrates the benchmark architecture. The contributions are summarized as follows:

- We propose NALMOBench, a new benchmark platform designed to evaluate text-to-EXE systems and LLMs for MODs, featuring over 600 paired natural language and executable database queries that cover both standard operations and real-world scenarios, categorized into 7 query types and stratified across 3 difficulty levels.
- We construct a corpus of approximately 6,000 moving object queries through a hybrid approach that combines manually collected NLQs with LLM-based automated data augmentation, incorporating semantic and formal verification.
- We further optimize the natural language understanding algorithm by constructing NLQ candidates and database candidates, respectively. We propose a divide-and-conquer paradigm to optimize structured language generation.
- We evaluate different NLIDB systems and LLMs on NALMOBench. Experimental results demonstrate that the baseline method achieves 68.96%, 60.07%, and 68.37 in translatability, translation accuracy, and translation efficiency score, respectively, outperforming existing approaches.

2 RELATED WORK

Benchmarks play a pivotal role in the development and validation of NLIDB systems. High-quality benchmarks can promote the vigorous development and continuous progress of NLIDB research. Early research on NLIDB benchmarks often utilize proprietary datasets, which are tailored to specific domains. For instance, parsing based systems like NaLIR, PRECISE [10], and ATHENA++ [11] rely on datasets such as Geoquery, MAS, and YELP, which focus on individual domains and lack generalizability, despite performing well in the respective domains.

With the development of neural network technology, research on NLIDB places higher requirements on system generalization. Although neural network-based technologies are emerging, a test dataset able to be used in a cross-domain context is missing. Thus, WikiSQL is proposed, which is the first large-scale multi-domain dataset for relational databases. WikiSQL contains 80,654 NLQs and 77,840 SQL statements. However, the SQL statements in WikiSQL are relatively simple, lacking complex operations such as sorting, grouping, and subqueries. Therefore, Spider further addresses the limitations of WikiSQL by constructing a large and complex NLIDB dataset for cross-domain semantic analyses. Spider contains 10,181 NLQs, 5,693 SQL statements, and involves more than 200 databases in 138 different domains. There are common simple SQL queries as well as complex ones such as nested queries. Although WikiSQL and Spider propose cross-domain datasets, the two benchmarks still emphasize on database schema and are insufficient to explore further on database values. KaggleDBQA [5] addresses the above problem by using databases extracted from real-world data sources, which are more relevant to real-world scenario applications.

With the arrival of the era of LLMs, the research of NLIDB has ushered in a new paradigm. Bird is proposed to evaluate LLMs. Bird contains 12,751 text-to-SQL pairs and 95 databases with a total size of 33.4 GB, spanning 37 professional domains. Bird proposes

a validation metric to evaluate the efficiency of NLIDB, though primarily focusing on time cost.

While Spider and Bird claim high generalization capabilities, the majority of these databases are specifically created by students, which may not adequately represent real-world database complexities. ScienceBenchmark [17] is the first text-to-SQL benchmark designed with complex real-world scientific databases. Real-world applications are likely to require high performance in a single domain, which may be more important than the ability to generalize across domains.

3 BENCHMARK CONSTRUCTION

Existing benchmarks are predominantly oriented towards text-to-SQL tasks, with no benchmarks specifically constructed for text-to-EXE scenarios. While claiming cross-domain applicability, these benchmarks fail to adequately address domain-specific requirements, such as moving object queries. Moreover, most benchmarks lack validation in real-world scenarios. To address these limitations, we develop NALMOBench, specifically designed for text-to-EXE validation in MODs. Algorithm 1 shows the workflow of the benchmark construction.

Algorithm 1: Corpus Construction and Annotation

Input : the list of n NLQs, Q ;
the schema information of the database, S ;
moving objects NLQ corpus, MOC ;
the list of n executable database queries, EL ;

Output : validated query pairs, P
initialize Q with collected NLQs

for $q_i \in Q$ **do**
 $el_i \leftarrow$ manual annotation of q_i
Models construction according to $Type(q_i)$
for $Model \in Models$ **do**
 $q \leftarrow Model(S)$
 $Q.append(q)$

for $q_i \in Q$ **do**
 if q_i fails to meet the semantic, syntax, and entity constraints **then**
 $Q.remove(q_i)$

for $q_i \in Q, el_i \in EL$ **do**
 $el_i \leftarrow NALMO(q_i)$
 if $z3(el_i) \wedge semantic(el_i) = semantic(q_i)$ **then**
 $EL.append(el_i)$

$P.append(Q, EL)$
return P

3.1 Corpus Construction Methodology

We develop a template-based methodology leveraging collected NLQs, augmented by LLMs, to construct a preprocessed corpus of 6,000 NLQs about moving objects, covering time interval query, range query, nearest neighbor query, join query, trajectory similarity query, cross-region query, and detour query.

3.1.1 Natural Language Query Collection. We collect and design 100 robust moving objects NLQs. For each type of query, except real-world queries, we collect and manually refine 80 NLQs from journals

翻译准确性和执行效率。图2展示了基准架构。贡献总结如下：

- 我们提出了 NALMOBench，一个新的基准平台，旨在评估文本到 EXE 系统和用于 MOD 的大型语言模型（LLM），该平台包含超过 600 个自然语言与可执行数据库查询的配对，涵盖标准操作和真实世界场景，分为 7 种查询类型，并按 3 个难度级别分层。
- 我们通过一种混合方法构建了大约 6,000 个移动对象查询的语料库，该方法结合了人工收集的自然语言查询（NLQs）和基于大语言模型（LLM）的自动数据增强，并结合了语义和形式验证。
- 我们通过分别构建 NLQ 候选和数据库候选来进一步优化自然语言理解算法。我们提出了一种分而治之的范式来优化结构化语言生成。
- 我们在 NAL-MOBench 上评估了不同的 NLIDB 系统和 LLM。实验结果表明，基线方法在可翻译性、翻译准确性和翻译效率得分方面分别达到 68.96%、60.07% 和 68.37%，优于现有方法。

2 相关工作

基准在 NLIDB 系统的开发和验证中起着关键作用。高质量的基准可以促进 NLIDB 研究的积极发展和持续进步。早期关于 NLIDB 基准的研究通常使用专有数据集，这些数据集针对特定领域量身定制。例如，基于解析的系统如 NaLIR、PRECISE [10] 和 ATHENA++ [11] 依赖于如 Geoquery、MAS 和 YELP 等数据集，这些数据集专注于单个领域，尽管在各自领域表现良好，但缺乏通用性。

随着神经网络技术的发展，关于自然语言到数据库接口（NLIDB）的研究对系统的泛化能力提出了更高的要求。尽管基于神经网络的技术不断涌现，但缺乏可在跨领域情境中使用的测试数据集。因此，提出了 WiKiSQL，这是第一个面向关系数据库的大规模多领域数据集。WiKiSQL 包含 80,654 条自然语言问题（NLQs）和 77,840 条 SQL 语句。然而，WiKiSQL 中的 SQL 语句相对简单，缺乏排序、分组和子查询等复杂操作。因此，Spider 通过构建一个用于跨领域语义分析的大型复杂 NLIDB 数据集，进一步解决了 WiKiSQL 的局限性。Spider 包含 10,181 条 NLQs、5,693 条 SQL 语句，并涉及 138 个不同领域中的 200 多个数据库。数据集中既有常见的简单 SQL 查询，也有嵌套查询等复杂查询。尽管 WiKiSQL 和 Spider 都提出了跨领域的数据集，这两个基准仍然强调数据库模式。

随着大型语言模型（LLM）时代的到来，自然语言接口数据库（NLIDB）的研究迎来了新的范式。Bird 被提出用来评估 LLM。Bird 包含 12,751 对文本到 SQL 的匹配对和 95 个数据库，总规模为 33.4 GB，涵盖 37 个专业领域。Bird 提出

一种用于评估自然语言到数据库系统（NLIDB）效率的验证指标，尽管主要关注时间成本。

虽然 Spider 和 Bird 宣称具有很高的泛化能力，但其中大多数数据库是由学生专门创建的，这可能无法充分代表现实世界的数据库复杂性。ScienceBenchmark [17] 是第一个设计用于复杂现实世界科学数据库的文本到 SQL 基准。现实世界的应用很可能需要在单一领域中实现高性能，这可能比跨领域的泛化能力更为重要。

3 基准建设

现有的基准测试主要面向文本到 SQL 的任务，没有专门为文本到 EXE 情境构建的基准测试。虽然声称具有跨领域适用性，但这些基准测试未能充分满足特定领域的需求，例如移动对象查询。此外，大多数基准测试缺乏在真实场景中的验证。为了解决这些限制，我们开发了 NALMOBench，专门用于 MOD 中的文本到 EXE 验证。算法 1 显示了基准测试构建的工作流程。

Algorithm 1: Corpus Construction and Annotation

```

Input : the list of  $n$  NLQs,  $Q$ ;
          the schema information of the database,  $S$ ;
          moving objects NLQ corpus,  $MOC$ ;
          the list of  $n$  executable database queries,  $EL$ ;
Output : validated query pairs,  $P$ 
          initialize  $Q$  with collected NLQs
for  $q_i \in Q$  do
   $el_i \leftarrow$  manual annotation of  $q_i$ 
  Models construction according to  $Type(q_i)$ 
for  $Model \in Models$  do
   $q \leftarrow Model(S)$ 
   $Q.append(q)$ 
for  $q_i \in Q$  do
  if  $q_i$  fails to meet the semantic, syntax, and entity
  constraints then
     $Q.remove(q_i)$ 
for  $q_i \in Q, el_i \in EL$  do
   $el_i \leftarrow NALMO(q_i)$ 
  if  $z3(el_i) \wedge semantic(el_i) = semantic(q_i)$  then
     $EL.append(el_i)$ 
   $P.append(Q, EL)$ 
return  $P$ 

```

3.1 语料库构建方法

我们开发了一种基于模板的方法，利用收集的自然语言查询（NLQs），并通过大语言模型（LLMs）增强，以构建一个包含 6,000 条关于移动对象的自然语言查询的预处理语料库，涵盖时间区间查询、范围查询、最近邻查询、连接查询、轨迹相似性查询、跨区域查询和绕行查询。

3.1.1 自然语言查询收集。我们收集并设计了 100 个稳健的运动物体自然语言查询（NLQs）。对于每种类型的查询，除真实世界查询外，我们从期刊中收集并手动完善了 80 个自然语言查询。

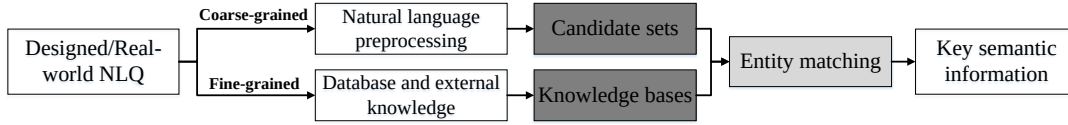


Figure 3: Natural language understanding.

and conferences. We suppose that queries sourced from top-tier database journals and conferences are of high quality. Real-world scenario queries are proposed by traffic enforcement officers and subsequently modified by experts, which reflect actual scenarios and are further fine-tuned by experts to align with the research context. Additionally, we manually annotate the 100 NLQs, and the detailed annotation process will be described later.

3.1.2 Automatic Augmentation. Building upon the collected queries, we develop advanced NLQ augmentation templates for each query type. Limited by space, we show one template of the range query: “Which <object> pass through <location_type> <specific_location> between <start_time> to <end_time>?”. Such templates serve as the foundation for constructing corresponding prompts using the one-shot methodology. We then leverage GPT-4o to expand the corpus, thereby enriching the expressive diversity of the NLQs. Corresponding prompts are given in Figure 4.

```

### Complete SECONDO executable language query only and with no explanation
### SECONDO executable language tables , with their properties :
#
#Schema:{schema information}
#

Formulate a {specific query type} query, filling the masked slots of the given template to generate the natural language query.
Template: Find the same trajectory between <object1> <object_id1/object_identification1> and <object2> <object_id2/object_identification2> during the time period [<start_time>, <end_time>].
Generated Queries:

```

Figure 4: Prompt of natural language generation with natural language templates.

3.1.3 Question Review. To further ensure the quality of the NLQs, we conduct a secondary validation process, using chain-of-thought (CoT) methodology to design prompts for GPT-4o to verify the following aspects of each query: (i) *Whether the syntax is correct.* (ii) *Whether the entity information corresponds to the query type.* (iii) *Whether there is any ambiguity.* Non-compliant queries are reviewed by experts. Since further NLQ extraction will be conducted during the benchmark construction process, the primary focus of the initial validation is to ensure the syntactic correctness and robustness of the query content, without interacting with the database information.

3.2 Benchmark Construction Methodology

We employ a hybrid approach combining manual and automated methods to construct NLQ and executable language pairs. Initially, we manually create 100 robust pairs of moving objects queries. Subsequently, we extract approximately 500 NLQs from an existing corpus using automated validation methods. In addition, we utilize the methods given in NALSpatial [8] for initial annotation, followed by validation of the annotated queries using both syntactic and

formal verification techniques. Finally, we rely on experts to label queries that cannot be automatically annotated.

3.2.1 Manual Generation. We engage three graduate students specializing in spatio-temporal databases and two traffic enforcement officers in the manual construction of query pairs. The verification for non-real-world scenarios is conducted by the three graduate students. Queries related to cross-region and detour operations are proposed by the enforcement officers. The graduate students then fine-tune these queries based on the database and construct the corresponding pairs, which are validated by the enforcement officers through executing the executable database queries. In total, 100 query pairs are constructed. Meanwhile, we utilize the constructed corpus to extract NLQs and use GPT-4o to automate the validation process through CoT with organized examples to extract the NLQs.

3.2.2 Annotation. We utilize the method provided by NALSpatial for initial annotation. However, the method can generate only part types of queries. So, we use manually construct pairs as examples and leverage the GPT-4o model to assess the semantic equivalence between generated executable languages and original NLQs. Subsequently, we conduct Z3 formal verification, which involves parsing the executable languages to extract the query types, structure, and components. Symbolic variables are created for entities, and conditions in different clause categories are converted into Z3 constraints to check constraint consistency, specifically to identify and resolve any contradictions within the query constraints. For queries that either can not be generated or fail the verification processes, we enlist the expertise of domain specialists to adjust or reconstruct the executable database queries. In particular, all real-world scenario queries are manually constructed.

4 NATURAL LANGUAGE UNDERSTANDING

The extensive semantics and diverse syntactic variations of natural language impede direct machine comprehension, leading to substantial difficulties in entity recognition and extraction. To address these challenges, we develop a sophisticated two-stage process, comprising coarse-grained and fine-grained natural language processing (NLP) modules, as depicted in Figure 3. In the initial coarse-grained stage, the system generates a candidate set of entities, including time, locations, objects, relations, moving object identifiers, and nearest neighbor numbers. Subsequently, in the fine-grained stage, a pre-constructed knowledge base is leveraged to prune the candidate entity set and extract precise entities.

4.1 Natural Language Candidate Entity

In this phase, our primary objective is the construction of the natural language candidate set. Efficiency is a key consideration, necessitating the rapid identification of entity information. The inherent

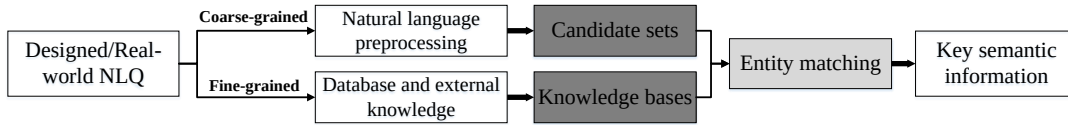


图 3：自然语言理解。

和会议。我们假设来自顶级数据库期刊和会议的查询是高质量的。现实场景中的查询由交通执法人员提出，随后由专家修改，这反映了实际场景，并由专家进一步微调以符合研究背景。此外，我们手动标注了这100个自然语言查询，详细的标注过程将在后文描述。

3.1.2 自动增强。在收集的查询基础上，我们为每种查询类型开发了先进的自然语言查询（NLQ）增强模板。由于篇幅限制，我们展示了范围查询的一个模板：“哪些 <对象> 会在 <位置类型> <特定位置> 之间从 <开始时间> 到 <结束时间> 经过？”这些模板作为使用单次示例（one-shot）方法构建相应提示（prompt）的基础。然后，我们利用 GPT-4o 扩展语料库，从而丰富 NLQ 的表达多样性。相应的提示如图4所示。

```

### 仅完成 SECONDO 可执行语言查询且不附带解释 ### SECONDO 可执行语言表格及其属性: ##模式:
{模式信息} #根据给定模板, 填写被掩盖的槽位以生成自然语言查询, 形成{特定查询类型}查询。模板:
在时间段 [<start_time>, <end_time>] 内, 找到 <object1> <object_id1/object_identification1> 与 <object2> <
object_id2/object_identification2> 之间的相同轨迹。生成的查询:
  
```

图4：使用自然语言模板进行自然语言生成的提示。

3.1.3 问题审查。为了进一步确保自然语言问题（NLQ）的质量，我们进行二次验证流程，使用链式思维（CoT）方法设计提示，让 GPT-4o 验证每个查询的以下方面：（i）语法是否正确。（ii）实体信息是否与查询类型对应。（iii）是否存在任何歧义。对于不合规的查询，由专家进行审查。由于在基准构建过程中还会进行进一步的 NLQ 提取，初步验证的主要重点是确保查询内容的语法正确性和稳健性，而不与数据库信息进行交互。

3.2 基准构建方法

我们采用结合手动与自动方法的混合方式来构建自然语言查询（NLQ）与可执行语言对。最初，我们手动创建了100对关于移动物体查询的稳健对。随后，我们使用自动验证方法从现有语料库中提取约500条NLQ。此外，我们利用 NALSpatial [8] 中给出的方法进行初始注释，然后使用语法和...对注释查询进行验证。

形式化验证技术。最后，我们依赖专家来标注无法自动注释的查询。

3.2.1 手动生成。我们聘请了三位专门研究时空数据库的研究生和两名交通执法人员参与查询对的手动构建。非现实场景的验证由这三位研究生进行。跨区域和绕行操作相关的查询由执法人员提出。研究生随后根据数据库对这些查询进行微调，并构建相应的查询对，这些查询对通过执行可执行数据库查询由执法人员进行验证。总共构建了100个查询对。同时，我们利用构建的语料库提取自然语言查询（NLQs），并使用 GPT-4o 通过带有组织示例的链式推理（CoT）自动化验证过程来提取 NLQs。

3.2.2 注释。我们使用 NALSpatial 提供的方法进行初始注释。然而，该方法只能生成部分类型的查询。因此，我们使用手工构建的示例对，并利用 GPT-4o 模型评估生成的可执行语言与原始自然语言查询（NLQs）之间的语义等价性。随后，我们进行 Z3 形式验证，这包括解析可执行语言以提取查询类型、结构和组成部分。为实体创建符号变量，并将不同子句类别中的条件转换为 Z3 约束，以检查约束的一致性，特别是用于识别和解决查询约束中的任何矛盾。对于无法生成或未通过验证的查询，我们会请领域专家调整或重建可执行数据库查询。特别地，所有现实场景的查询均为手工构建。

4 自然语言理解

自然语言的广泛语义和多样的句法变体阻碍了机器的直接理解，导致实体识别和提取方面存在重大困难。为了解决这些挑战，我们开发了一个复杂的两阶段处理流程，包括粗粒度和细粒度的自然语言处理（NLP）模块，如图 3 所示。在初始的粗粒度阶段，系统生成一组候选实体，包括时间、地点、对象、关系、移动对象标识符和最近邻数量。随后，在细粒度阶段，利用预先构建的知识库对候选实体集进行修剪，并提取精确实体。

4.1 自然语言候选实体

在这个阶段，我们的主要目标是构建自然语言候选集。效率是一个关键考虑因素，这需要快速识别实体信息。固有的

complexity of semantic structures necessitates the use of advanced NLP tools to approximate and locate potential entities.

Considering the imperative for swift processing, we select spaCy as the primary NLP tool, with NLTK as a potential alternative. Our framework initial implementation predominantly relies on the robust tokenization and named entity recognition capabilities of spaCy. Employing spaCy, we construct two principal sets: (i) *the information set* and (ii) *the number set*. The information set encompasses candidate locations, objects, and relations. The number set comprises the number of nearest neighbors, moving object identifiers, and time information.

Given that the number set encapsulates numerical and temporal information, we can precisely locate time and some of the numbers. For instance, when encountering a time expression containing a colon, such as “8:00 o’clock”, we utilize a regular expression to achieve accurate matching and extraction. For hour expressions lacking a colon, such as “8am”, we also employ a regular expression for matching and extraction. The number of nearest neighbors can then be determined according to the semantic distance of the keywords like “nearest”. Due to the potential ambiguity of the query object, object identification necessitates further validation.

4.2 Database Candidate Entity

Aligning natural language entity information with deterministic database information presents significant challenges due to inherent ambiguities and uncertainties in natural language. To address this, we propose the construction of database candidates for precise entity extraction. To achieve accurate extraction of location, objects, relations, and object identifiers, we propose a sophisticated moving objects knowledge base (MOKB). MOKB is composed of two sub-knowledge bases: (i) *the relation knowledge base (RKB)* and (ii) *the location knowledge base (LKB)*.

We conduct a thorough analysis of the objects within the database to identify relations containing the *mpoint* attribute, which is utilized to represent moving objects. This relation encapsulates the target moving objects. The LKB primarily extracts objects with attributes such as *point*, *line*, and *region* from the database. Recognizing the complexities posed by synonyms, abbreviations, and ambiguous terms for locations, we leverage the capabilities of GPT-4o to augment the existing LKB with potential synonyms, abbreviations, and ambiguous expressions. The extended terms are mapped to the original locations solely for entity matching and will not be manifested as final location entities. The RKB is employed to determine moving objects relations and objects. Once an object is identified, the nearest numerical value in semantic distance from the object, existing within the number set and not previously identified as another entity, is ascertained as the object identifier. The location information can then be retrieved from the LKB.

When managing large-scale databases, the knowledge base can expand significantly, leading to prohibitive time consumption during retrieval processes. To mitigate this, we devise a hierarchical index for MOKB to expedite retrieval operations.

5 STRUCTURED LANGUAGE GENERATION

The inherent complexity of moving object queries renders the development of a universal structured language model (SLM) impractical

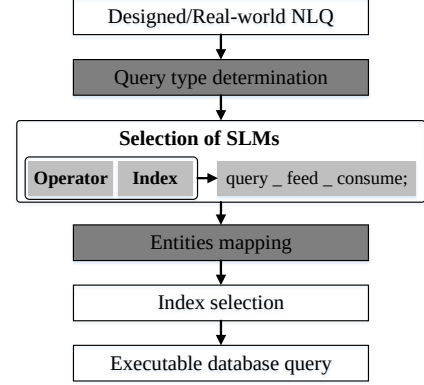


Figure 5: Structured language construction.

and costly, especially in real-world scenarios. Therefore, we propose a divide-and-conquer strategy to reduce model design costs and develop a query optimization model that refines generated structured languages through index selection, with the detailed construction process illustrated in Figure 5.

5.1 Structured Language Model Sketch and Query Type Classification

Figure 5 delineates the sketch SLM, where $\langle k \rangle$, $\langle t \rangle$, $\langle o_i \rangle$, $\langle place \rangle$, and $\langle relation \rangle$ denote the undetermined number of nearest neighbors, time, object and identifier, location, and moving objects relation, respectively. In scenarios where a location is encapsulated within a moving objects relation, the $\langle place \rangle$ placeholder in the model is substituted with the following expression. Here, $\langle tmp relation \rangle$ and $\langle name \rangle$ represent the moving objects relation and the name of the object: “ $\langle tmp relation \rangle$ feed filter [$.Name = \langle name \rangle$] extract [Trip]”. Table 1 enumerates part of the key operators, encompassing names, signatures, and functionalities.

Table 1: Operators in SECONDO.

Operator	Signature	Meaning
filter	$stream \times filter \ condition \rightarrow stream$	Filter the elements of the stream by a predicate.
trajectory	$mpoint \rightarrow line$	Map continuous moving points into trajectories.
deftime	$mpoint \rightarrow periods$	Project motion onto the time dimension.
at	$mpoint \times \{point, region\} \rightarrow mpoint$	Return moving points to a certain point or region.
knearest	$stream \times mpoint \times int \ k \rightarrow stream$	Compute the k nearest neighbors.
dist_euclidean	$pointseq \times pointseq \rightarrow real$	Compute the trajectory similarity.

For the type classification model training, the corpus is partitioned into a training set and a test set with a ratio of 8:2. LSTM is employed for training and the trained model can be utilized to determine the query type of the input NLQ.

5.2 Structured Language Model Selection and Slot Filling

For different query types, we map the corresponding entities to respective slots using predefined mapping rules and combine them

语义结构的复杂性需要使用先进的自然语言处理工具来近似和定位潜在的实体。

考虑到快速处理的必要性，我们选择 spaCy 作为主要的自然语言处理工具，NLTK 作为潜在的替代方案。我们框架的初始实现主要依赖于 spaCy 强大的分词和命名实体识别能力。使用 spaCy，我们构建了两个主要集合：(i) 信息集合 和 (ii) 数字集合。信息集合包含候选位置、对象和关系。数字集合包括最近邻数量、移动对象标识符和时间信息。

鉴于数字集合包含数值和时间信息，我们可以精确地定位时间和部分数字。例如，当遇到包含冒号的时间表达式，如“8:00”，我们使用正则表达式进行精确匹配和提取。对于缺少冒号的小时表达式，如“8am”，我们也使用正则表达式进行匹配和提取。最近邻的数量可以根据“最近”等关键词的语义距离来确定。由于查询对象可能存在模糊性，对象识别需要进一步验证。

4.2 数据库候选实体

将自然语言实体信息与确定性的数据库信息对齐存在重大挑战，因为自然语言本身存在固有的歧义和不确定性。为了解决这一问题，我们提出构建数据库候选项以实现精确的实体提取。为了实现位置、对象、关系和对象标识符的准确提取，我们提出了一个复杂的移动对象知识库（MOKB）。MOKB 由两个子知识库组成：(i) 关系知识库（RKB）和 (ii) 位置知识库（LKB）。

我们对数据库中的对象进行全面分析，以识别包含 mpoint 属性的关系，该属性用于表示运动对象。该关系封装了目标运动对象。LKB 主要从数据库中提取具有点、线和区域等属性的对象。鉴于同义词、缩写和位置歧义术语带来的复杂性，我们利用 GPT-4o 的能力来增强现有 LKB，添加潜在的同义词、缩写和歧义表达。扩展的术语仅用于与原始位置进行实体匹配，并不会作为最终位置实体呈现。RKB 用于确定运动对象的关系和对象。一旦识别出对象，将从数字集中确定与该对象语义距离最近且之前未被识别为其他实体的数值作为对象标识符。

在管理大规模数据库时，知识库可能显著扩展，导致检索过程中耗时过长。为缓解这一问题，我们为MOKB设计了一个分层索引以加快检索操作。

5 结构化语言生成

移动对象查询的固有复杂性使得开发通用结构化语言模型（SLM）不切实际

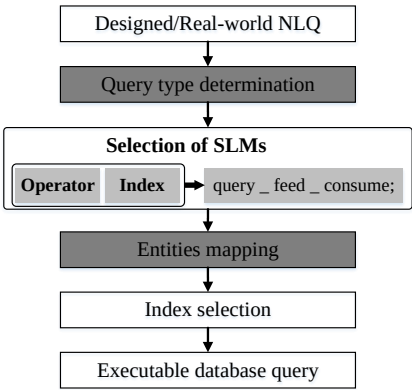


图5：结构化语言构建。

并且代价高，尤其是在现实场景中。因此，我们提出了一种分而治之的策略以降低模型设计成本，并开发了一种查询优化模型，通过索引选择来优化生成的结构化语言，其详细构建过程如图5所示。

5.1 结构化语言模型概要与查询类型分类

图 5 描述了草图 SLM，其中 $\langle k \rangle$ 、 $\langle t \rangle$ 、 $\langle o_i \rangle$ 、 $\langle place \rangle$ 和 $\langle relation \rangle$ 分别表示未确定数量的最近邻、时间、对象和标识符、位置以及移动对象关系。在一个位置包含在移动对象关系中的场景中，模型中的 $\langle place \rangle$ 占位符会被以下表达式替换。这里， $\langle tmp relation \rangle$ 和 $\langle name \rangle$ 分别表示移动对象关系和对象名称：“ $\langle tmp relation \rangle$ feed filter [$.Name = \langle name \rangle$] extract [Trip]”。表 1 列举了部分关键操作符，包括名称、签名和功能。

表1：SECONDO中的操作符。

Operator	Signature	Meaning
filter	$stream \times filter \ condition \rightarrow stream$	Filter the elements of the stream by a predicate.
trajectory	$mpoint \rightarrow line$	Map continuous moving points into trajectories.
deftime	$mpoint \rightarrow periods$	Project motion onto the time dimension.
at	$mpoint \times \{point, region\} \rightarrow mpoint$	Return moving points to a certain point or region.
knearest	$stream \times mpoint \times int \ k \rightarrow stream$	Compute the k nearest neighbors.
dist_euclidean	$pointseq \times pointseq \rightarrow real$	Compute the trajectory similarity.

对于类型分类模型的训练，语料库被按照8:2的比例划分为训练集和测试集。LSTM被用于训练，训练好的模型可以用来确定输入自然语言查询（NLQ）的查询类型。

5.2 结构化语言模型选择与槽位填充

对于不同的查询类型，我们使用预定义的映射规则将相应的实体映射到各自的槽位并将它们组合起来

with the required operators to generate the structured languages. SLMs for time interval queries, range queries, and nearest neighbor queries can be constructed based on methodologies described in NALMO [13]. In this paper, we focus on the similarity threshold, usage of indexes, and construction of structured languages in real-world scenarios. Limited by space, we just take the cross-region query as an example. Detailed description of SLMs are given in NALSpatial and NALMO.

To determine whether a taxi trajectory intersects with the operational boundary and within the operational area, we use the *intersects* operator. If there is an intersection or the trajectory is outside the operational area, we deduce the existence of a cross-region operation. However, if the taxi exits the operational boundary and then re-enters within a short period, this behavior will not be considered as a cross-region operation.

6 EXPERIMENTAL RESULTS AND DISCUSSION

We report extensive experimental results in this section. The implementation is developed in a computer (Intel(R) Core(TM) i9-10850K CPU, 3.60 GHz, 32 GB memory, 512 GB disk) running Ubuntu 20.04 (64 bits, kernel version 5.14.0-1051-oem). We aim to investigate the performance of existing text-to-SQL methods and LLMs in addressing the text-to-EXE problem, with a focus on their applicability and robustness in the domain of MODs. To ensure a comprehensive evaluation, we not only assess the translation accuracy but also the quality of the generated executable languages, including translation efficiency and execution efficiency. We provide a thorough understanding of the strengths and weaknesses of different approaches, offering valuable insights for real-world applications.

6.1 Experimental Setup

Setting. As described in Section 3, a combined approach of manual and LLM-based annotation is employed. Although leveraging LLMs for generation can reduce workload, a certain degree of manual review remains necessary. Consequently, 600 pairs of NLQs and structured query pairs are ultimately annotated. Example pairs¹ are available. Existing queries primarily fall into two categories: (i) *those specifically designed for databases* and (ii) *those tailored to real-world requirements*, which both are considered. The specialized queries include time interval queries (TI), range queries (RA), nearest neighbor queries (NN), join queries (J), and trajectory similarity queries (TS), while real-world scenario queries encompass cross-region queries (CR) and detour queries (DT). According to the query length, SQL query difficulty is categorized into four levels, including simple, medium, hard, and extra hard in Spider. Building upon this, we further classify moving object queries into three complexity levels, including simple, medium, and hard, by incorporating their intricate query types, shown in Table 3.

Datasets. The experiment utilize datasets comprising both real-world taxis, cars, and buses movement trajectories and system-simulated train movement trajectories, as shown in Table 2, ensuring the comprehensiveness and accuracy of our experimental results. The real-world trajectory dataset *nanjingtest* spans the period from 00:00 to 24:00 on June 15, 2024, capturing movement fragments from 302 moving objects, containing a total of 917,006

Table 2: Details of the datasets used in the benchmark.

Dataset	#Points	#Lines	#Regions	#Moving objects
nanjingtest	9000	887	13	302
berlintest	3040	4078	330	562

moving object points. Meanwhile, the system-simulated train trajectory dataset *berlintest* covers the period from 06:00 to 09:00 on November 20, 2020, involving trajectory fragments from 562 trains and consisting of 51,544 moving object points.

6.2 Evaluation Metrics

In the domain of text-to-SQL, the evaluation of system performance mainly hinges upon three critical metrics: (i) *Exact Match (EM)*, (ii) *Execution Accuracy (EX)*, and (iii) *Valid Efficiency Score (VES)* [7]. Our evaluation metrics are inspired by the three metrics above.

To evaluate the natural language understanding capabilities of the text-to-EXE systems, we propose the metric of Translatability (TA). Given the set of executable database queries generated by the system and the set of input NLQs, denoted by EQ and N , respectively, TA is defined as:

$$TA = \frac{|EQ|}{|N|} \quad (1)$$

This metric is used to assess the semantic completeness of executable languages, which are both logically correct and entity-complete. TA broadens the scope by acknowledging all structurally viable database queries, provided executable queries incorporate the requisite entities and maintain logical integrity.

Furthermore, to evaluate the proficiency in formulating structured languages, we propose the Translation Precision (TP) metric. Defined over the set of executable database queries that align with expected results and the input NLQs N , TP assesses whether executable languages deliver the expected outputs post-execution. We compare the results between the generated executable database query and the golden one, assuming the same execution results indicate equivalence. Let S_n be the executable database query set for the n -th example, G_n be the golden database query set for the n -th example, $R(S_n)$ be the result set after executing S_n , $R(G_n)$ be the result set after executing G_n , TP can be defined as:

$$TP = \frac{1}{N} \sum_{n=1}^N (\mathbb{1}(S_n, G_n) \cdot \mathbb{1}(R(S_n), R(G_n))) \quad (2)$$

where $\mathbb{1}(A, B)$ is an indicator function defined as:

$$\mathbb{1}(A, B) = \begin{cases} 1, & \text{if } A = B \\ 0, & \text{otherwise} \end{cases} \quad (3)$$

To comprehensively assess the quality of the generated executable languages, our analysis extends to encompass efficiency metrics, called Translation Efficiency Score (TES), including execution time and a composite measure incorporating CPU time during execution. TES goes beyond VES by considering execution time and CPU time, thus providing a more comprehensive assessment dimension. We only consider statements that are capable of producing the same results as the given executable languages. Let $E(\cdot)$ represent the measurement of execution time and $C(\cdot)$ represent the

¹<https://pan.nuaa.edu.cn/share/c9bd35cbe21c175137d5de4db9>.

使用所需的操作符来生成结构化语言。基于NALMO [13] 中描述的方法，可以构建用于时间区间查询、范围查询和最近邻查询的SLM。在本文中，我们重点关注相似性阈值、索引的使用以及实际场景中结构化语言的构建。由于篇幅有限，我们仅以跨区域查询为例。SLM的详细描述见NALSpatial和NALMO。

为了确定出租车轨迹是否与运营边界相交以及是否位于运营区域内，我们使用 `intersects` 操作符。如果存在相交或轨迹位于运营区域之外，我们可以推断存在跨区域运营。然而，如果出租车在短时间内离开运营边界后又重新进入，这种行为不会被视作跨区域运营。

6 实验结果与讨论

我们在本节中报告了大量实验结果。实现是在一台运行 Ubuntu 20.04 (64 位, 内核版本 5.14.0-1051-oem) 的计算机上开发的, 该计算机配备 Intel(R) Core(TM) i9-10850K CPU, 3.60 GHz, 32 GB 内存, 512 GB 硬盘。我们的目标是研究现有的文本到 SQL 方法和大型语言模型在解决文本到 EXE 问题中的表现, 重点关注它们在 MODs 领域中的适用性和鲁棒性。为了确保全面评估, 我们不仅评估翻译准确性, 还评估生成的可执行语言的质量, 包括翻译效率和执行效率。我们提供了对不同方法优缺点的深入理解, 为实际应用提供有价值的见解。

6.1 实验设置

设置。如第3节所述, 采用手动和基于大语言模型 (LLM) 的注释相结合的方法。虽然利用LLM进行生成可以减轻工作量, 但仍然需要一定程度的人工审查。因此, 最终对600对自然语言查询 (NLQs) 和结构化查询对进行了注释。示例对1可用。现有查询主要分为两类: (i) 专门为数据库设计的查询, (ii) 适用于现实世界需求的查询, 这两类均被考虑。专用查询包括时间间隔查询 (TI)、范围查询 (RA)、最近邻查询 (NN)、连接查询 (J) 和轨迹相似性查询 (TS), 而现实场景查询则包括跨区域查询 (CR) 和绕行查询 (DT)。根据查询长度, Spider将SQL查询难度分为四个级别, 包括简单、中等、困难和极难。在此基础上, 我们进一步将移动对象查询分为三类。

数据集。本实验使用的数据集包括真实世界的出租车、汽车和公共汽车移动轨迹, 以及系统模拟的火车移动轨迹, 如表2所示, 以确保实验结果的全面性和准确性。真实世界的轨迹数据集nanjingtest覆盖了2024年6月15日00:00至24:00的时间段, 记录了302个移动对象的移动片段, 总共包含917,006条数据。

表 2: 基准测试中使用的数据集详情。

Dataset	#Points	#Lines	#Regions	#Moving objects
nanjingtest	9000	887	13	302
berlintest	3040	4078	330	562

移动目标点。与此同时, 系统模拟的列车轨迹数据集berlintest覆盖了2020年11月20日06:00到09:00的时间段, 涉及来自562列火车的轨迹片段, 共包含51,544个移动目标点。

6.2 评估指标

在文本到 SQL 的领域中, 系统性能的评估主要依赖于三个关键指标: (i) 完全匹配 (EM), (ii) 执行准确率 (EX), 以及 (iii) 有效效率评分 (VES) [7]。我们的评估指标受到上述三个指标的启发。

为了评估文本到可执行文件系统的自然语言理解能力, 我们提出了可翻译性 (TA) 这一指标。给定系统生成的可执行数据库查询集合和输入自然语言问题集合, 分别记为 EQ 和 N, TA 定义如下:

$$TA = \frac{|EQ|}{|N|} \quad (1)$$

该指标用于评估可执行语言的语义完整性, 这些语言在逻辑上正确且实体完整。TA通过承认所有结构上可行的数据库查询来扩大范围, 前提是可执行查询包含所需实体并保持逻辑完整性。

此外, 为了评估在构建结构化语言方面的熟练程度, 我们提出了翻译精确度 (TP) 指标。在与预期结果和输入自然语言查询 (NLQs) N 对齐的可执行数据库查询集合上定义, TP 评估可执行语言在执行后是否能够产生预期输出。我们比较生成的可执行数据库查询与黄金查询的结果, 假设相同的执行结果表示两者等价。设 S_n 为第 n 个示例的可执行数据库查询集合, G_n 为第 n 个示例的黄金数据库查询集合, $R(S_n)$ 为执行 S_n 后的结果集, $R(G_n)$ 为执行 G_n 后的结果集, 则 TP 可以定义为:

$$TP = \frac{1}{N} \sum_{n=1}^N (\mathbb{1}(S_n, G_n) \cdot \mathbb{1}(R(S_n), R(G_n))) \quad (2)$$

其中 $\mathbb{1}(A, B)$ 是定义如下的指示函数:

$$\mathbb{1}(A, B) = \begin{cases} 1, & \text{if } A = B \\ 0, & \text{otherwise} \end{cases} \quad (3)$$

为了全面评估生成的可执行语言的质量, 我们的分析扩展到包括称为翻译效率得分 (Translation Efficiency Score, TES) 的效率指标, 包括执行时间和在执行过程中 CPU 时间的综合测量。TES 超越了 VES, 通过考虑执行时间和 CPU 时间, 从而提供更全面的评估维度。我们只考虑能够产生与给定可执行语言相同结果的语句。令 $E(\cdot)$ 表示执行时间的测量, $C(\cdot)$ 表示

¹<https://pan.nuaa.edu.cn/share/c9bd35cbe21c175137d5de4db9>.

Table 3: Results of TA and TP.

Method	TA/%								TP/%							
	Easy		Medium			Hard		All	Easy		Medium			Hard		All
	TI	RA	NN	J	TS	CR	DT		TI	RA	NN	J	TS	CR	DT	
NaLIR	5.84	3.06	1.79	1.05	2.00	0.00	0.00	2.52	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
ATHENA++	37.27	31.61	21.43	27.37	28.00	10.17	23.53	29.87	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
ValueNet	35.04	20.41	19.64	18.00	16.95	11.86	15.69	24.33	16.79	14.29	10.71	5.26	2.00	3.39	5.88	9.23
LLaMA3-70B (few-shot)	51.10	33.16	30.17	22.11	16.50	10.17	31.37	30.20	23.36	19.47	20.54	10.00	5.50	5.93	6.86	14.18
GPT-4o (few-shot)	68.98	44.75	45.54	30.53	35.50	11.87	37.26	42.87	47.45	34.19	35.72	20.53	9.50	7.63	18.65	27.01
NALMO+	89.05	79.78	78.57	73.68	77.00	45.76	58.82	68.96	72.99	61.22	57.14	61.05	65.00	37.29	41.18	60.07

TI, Time Interval Query; RA, Range Query; NN, Nearest Neighbor Query; J, Join Query; TS, Trajectory Similarity Query; CR, Cross-Region; DT, Detour.

measurement of CPU time. The two functions are used to measure the absolute time in a given environment. We can design a formula that combines these two functions:

$$TES = \frac{\sum_{n=1}^N (\mathbb{1}(R(S_n), R(G_n)) \cdot \text{Cost}(Y_n, \hat{Y}_n))}{N} \quad (4)$$

$$\text{Cost}(Y_n, \hat{Y}_n) = \frac{\sqrt{E(Y_n)}}{\sqrt{E(\hat{Y}_n)}} \cdot \frac{\sqrt{C(Y_n)}}{\sqrt{C(\hat{Y}_n)}} \quad (5)$$

where $\text{Cost}(\cdot)$ denotes the relative efficiency of the generated executable languages compared to the standard executable languages. We comprehensively consider the execution time and CPU time ratio of the standard executable languages versus the generated executable languages to ensure a thorough evaluation.

6.3 Baseline Methods

We conduct a comparative analysis of six systems. Given the rapid advancements in text-to-SQL, we select representative methods from three popular categories: (i) *Rule-based methods*. We employ NaLIR and ATHENA++. NaLIR is a classical rule-based approach that utilizes parse trees for entity extraction and alignment, and constructs SQL with user feedback. ATHENA++ utilizes OQL queries as an intermediate representation through a two-phase generation method. (ii) *Neural network-based methods*. We choose ValueNet [1], which builds upon IRNet by extending the SemQL grammar to include values in the generated queries. (iii) *LLM-based methods*. We leverage GPT-4o and the open-source model LLaMA3 as comparative methods. In the BIRD leaderboard, over half of the top ten methods are based on GPT-4o. LLaMA3 is considered, which is one of the most widely used open-source models. We design experiments using the methodology of DAIL-SQL [2], which integrates schema information into prompt design and example organization, utilizing few-shot methods. Additionally, we introduce the optimized method called NALMO+ for comparison. NALMO+ is a hybrid architecture integrated into the SECONDO system as an algebra module, and the corresponding operator is also developed.

6.4 Experimental Results

We consider the query translation capabilities. Table 3 presents TA and TP of all methods, providing detailed data for different levels of difficulty and query types. Since the executable languages in the MOD differ in both syntax structure and semantic representation

from standard SQL, rule-based methods focus solely on the natural language understanding process.

NaLIR constructs a semantic parse tree and then maps the nodes onto SQL fragments. This database preset approach yields high error rates in entity recognition and operator selection. Consequently, NaLIR acquires low TA, and TP cannot be directly measured. ATHENA++ combines a domain ontology with Stanford CoreNLP for advanced NLP in simple queries. simple queries involving few entities. However, ATHENA++ fails to extract or map fuzzy entities, such as synonyms, abbreviations, or ambiguous terms. In real-world scenario queries, where natural language often provides minimal entity information and multi-step decomposition is required, ATHENA++ breaks down entirely.

ValueNet emits an intermediate representation (IR), making the post-processing step that converts IR into structured SQL critical. By ingesting actual database contents, ValueNet matches the TA of CoreNLP-based methods. Yet, because ValueNet depends on large-scale supervised training, the ability to generate executable languages remains limited. TP stays below 20% across all categories, and declines further as query complexity increases.

To analyze the performance of LLMs, we design few-shot prompting experiments. Due to the syntactic and semantic gap between moving objects executable languages and standard SQL, LLMs without example prompts fail all query tasks. GPT-4o outperforms LLaMA3 in both TA and TP, showing over 50% and 100% relative improvements, respectively. However, the rich semantic patterns and lack of fixed operator entity templates make single-shot prompts insufficient for unseen queries, while the generated logic may be correct, the omission of an initial temporal filter step leads to slower execution than the golden executable database query. Increasing to five examples yields over 20% gains in both TA and TP for LLaMA3, and over 40% gains for GPT-4o. However, further increasing sample size may risk hallucinations, including operator misuse and the invention of non-existent operators.

For TES, both the small model of ValueNet and LLMs exhibit sub-optimal performance. ValueNet, LLaMA3, and GPT-4o achieve 12.43, 26.37, and 60.71, respectively. Due to the inherent white-box nature of executable languages, query efficiency can be directly controlled. Small models are tightly bound to the training datasets, and LLMs have not been exposed to enough training data directly related to database indexing, especially in terms of optimizing queries with indexes. This requires substantial re-training or fine-tuning with

Table 3: Results of TA and TP.

Method	TA/%								TP/%							
	Easy	Medium				Hard		All	Easy	Medium				Hard		All
	TI	RA	NN	J	TS	CR	DT		TI	RA	NN	J	TS	CR	DT	
NaLIR	5.84	3.06	1.79	1.05	2.00	0.00	0.00	2.52	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
ATHENA++	37.27	31.61	21.43	27.37	28.00	10.17	23.53	29.87	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
ValueNet	35.04	20.41	19.64	18.00	16.95	11.86	15.69	24.33	16.79	14.29	10.71	5.26	2.00	3.39	5.88	9.23
LLaMA3-70B (few-shot)	51.10	33.16	30.17	22.11	16.50	10.17	31.37	30.20	23.36	19.47	20.54	10.00	5.50	5.93	6.86	14.18
GPT-4o (few-shot)	68.98	44.75	45.54	30.53	35.50	11.87	37.26	42.87	47.45	34.19	35.72	20.53	9.50	7.63	18.65	27.01
NALMO+	89.05	79.78	78.57	73.68	77.00	45.76	58.82	68.96	72.99	61.22	57.14	61.05	65.00	37.29	41.18	60.07

TI, 时间间隔查询; RA, 范围查询; NN, 最近邻查询; J, 连接查询; TS, 轨迹相似性查询; CR, 跨区域; DT, 绕行。

CPU 时间的测量。这两个函数用于测量给定环境中的绝对时间。我们可以设计一个结合这两个函数的公式：

$$TES = \frac{\sum_{n=1}^N \left(\mathbb{1}(R(S_n), R(G_n)) \cdot \text{Cost}(Y_n, \hat{Y}_n) \right)}{N} \quad (4)$$

$$\text{Cost}(Y_n, \hat{Y}_n) = \frac{\sqrt{E(Y_n)}}{\sqrt{E(\hat{Y}_n)}} \cdot \frac{\sqrt{C(Y_n)}}{\sqrt{C(\hat{Y}_n)}} \quad (5)$$

其中 $\text{Cost}(\cdot)$ 表示生成的可执行语言相对于标准可执行语言的相对效率。我们全面考虑标准可执行语言与生成的可执行语言的执行时间和 CPU 时间比，以确保全面评估。

6.3 基线方法

我们对六个系统进行了比较分析。考虑到文本到SQL的快速发展，我们从三类流行方法中选择了具有代表性的方法：(i) 基于规则的方法。我们采用了NaLIR和ATHENA++。NaLIR是一种经典的基于规则的方法，利用解析树进行实体提取和对齐，并通过用户反馈构建SQL。ATHENA++通过两阶段生成方法，将OQL查询作为中间表示。(ii) 基于神经网络的方法。我们选择ValueNet [1]，它在IRNet的基础上扩展了SemQL语法，使生成的查询中包含值。(iii) 基于大语言模型的方法。我们利用GPT-4o和开源模型LLaMA3作为对比方法。在BIRD排行榜中，前十名的方法中有一半上都基于GPT-4o。考虑到LLaMA3是最广泛使用的开源模型之一。我们设计实验时使用了DAIL-SQL [2]的方法论，将模式信息集成到提示设计中。

6.4 实验结果

我们考虑查询翻译能力。表3展示了所有方法的TA和TP，提供了不同难度级别和查询类型的详细数据。由于MOD中的可执行语言在语法结构和语义表示上都有所不同

在标准 SQL 中，基于规则的方法仅专注于自然语言理解过程。

NaLIR 构建语义解析树，然后将节点映射到 SQL 片段。这种数据库预设方法在实体识别和操作符选择上产生高错误率。因此，NaLIR 获得较低的 TA，并且 TP 无法直接测量。ATHENA++ 结合了领域本体和 Stanford CoreNLP，用于简单查询的高级 NLP。简单查询涉及的实体较少。然而，ATHENA++ 无法提取或映射模糊实体，例如同义词、缩写或含糊术语。在实际场景查询中，自然语言通常提供的实体信息有限，并且需要多步骤分解，ATHENA++ 会完全失效。

ValueNet 发出中间表示 (IR)，这使得将 IR 转换为结构化 SQL 的后处理步骤显得至关重要。通过摄取实际数据库内容，ValueNet 达到了基于 CoreNLP 方法的 TA 水平。然而，由于 ValueNet 依赖大规模监督训练，其生成可执行语言的能力仍然有限。TP 在所有类别中均低于 20%，并且随着查询复杂性的增加而进一步下降。

为了分析大型语言模型 (LLMs) 的性能，我们设计了少量示例提示实验。由于运动对象可执行语言与标准 SQL 之间存在句法和语义差距，没有示例提示的 LLM 在所有查询任务中都失败。GPT-4o 在 TA 和 TP 上均优于 LLaMA3，分别显示出超过 50% 和 100% 的相对提升。然而，由于丰富的语义模式和缺乏固定的操作符实体模板，单次提示不足以应对未见过的查询，尽管生成的逻辑可能是正确的，但省略初始时间过滤步骤会导致执行速度比黄金可执行数据库查询慢。将示例数量增加到五个，为 LLaMA3 带来 TA 和 TP 超过 20% 的增益，为 GPT-4o 带来超过 40% 的增益。然而，进一步增加样本量可能会存在幻觉风险，包括操作符误用和虚构不存在的操作符。

对于 TES，ValueNet 的小模型和大语言模型 (LLMs) 都表现出次优性能。ValueNet、LLaMA3 和 GPT-4o 的得分分别为 12.43、26.37 和 60.71。由于可执行语言具有固有的白盒特性，查询效率可以被直接控制。小模型与训练数据集紧密绑定，而大语言模型没有接触过足够直接与数据库索引相关的训练数据，尤其是在使用索引优化查询方面。这需要大量的重新训练或微调。

large datasets. In contrast, the hybrid approach NALMO+ demonstrates a notable advantage, maintaining relatively efficient queries, achieving 68.37 of TES even in the absence of an extensive dataset.

In conclusion, these findings underscore the challenges posed by the NALMOBench benchmark, further reinforcing that text-to-EXE research is still in its early stages, with considerable potential for further exploration and advancement in the field.

6.5 Discussion of the Experiments

Query complexity is fundamentally governed by the intrinsic difficulty of the query. For simple TI queries, which only incorporate temporal constraints, target objects, and relation information. The structured languages are relatively rigid and utilize exclusively temporal operators. Consequently, both the natural language understanding phase and the structured language generation phase remain straightforward, enabling all evaluated methods to achieve highest scores in TA and TP. By contrast, moderate complexity queries, including RA, NN, J, and TS queries, entail a far richer set of entity attributes (e.g., time, locations, relations, object identifiers, and nearest neighbor numbers). This enriched semantic scope significantly heightens the difficulty of natural language parsing. Concurrently, the structured language grammar must cover an expanded operator vocabulary, causing a combinatorial explosion in operator and entity pairings. LLMs exhibit three principal shortcomings in this situation: (i) *Entity mapping errors*. LLMs struggle to perform precise date transformations and to resolve fuzzy spatial references, resulting in misidentified temporal or locational entities. (ii) *Operator selection failures*. When confronted with unseen operators, LLMs either omit necessary operators or fabricate non-existent ones, thereby rendering the generated queries syntactically invalid. (iii) *Module omissions*. LLMs often neglect critical sub-modules required for complete structured language generation, culminating in queries that cannot be executed.

Consider the TS query as a concrete example. The similarity threshold must be empirically calibrated on each dataset, demanding an operation that depends on dataset-specific experiments and manual adjustment. However, LLMs cannot infer optimal threshold values and frequently generate spurious operators or default to seldom used but valid ones, which degrades query performance. At the hard level, CR and DT queries typically involve multiple sequential processing steps or nested query constructs. Because real-world execution requires iterative query handling, existing approaches falter, with both TA and TP metrics remain below 50%.

7 CONCLUSION

In this paper, we introduce NALMOBench, a novel benchmark designed to evaluate both the capability and quality of translating NLQs into executable languages for MODs. NALMOBench covers not only synthetic moving objects datasets but also real-world trajectory data. We further present an LLM-based approach for generating a domain-specific database corpus. This method encompasses both routine moving objects queries and those arising from intricate real-world scenarios. To tackle the text-to-EXE problem, we decompose the problem into two sub-problems: (i) *natural language understanding* and (ii) *structured language generation*, and provide baseline algorithms for each stage.

Our experiments demonstrate that NALMOBench poses significant challenges to existing text-to-EXE methods and LLMs. We contend that NALMOBench establishes a rigorous new standard for evaluating text-to-EXE systems in the MOD field. NALMOBench lays the groundwork for addressing the inherent complexities of the usage of real-world trajectory data and for advancing research into optimizing the efficiency of post-translated executable languages.

8 ACKNOWLEDGMENT

This work was supported by Postgraduate Research & Practice Innovation Program of Jiangsu Province (KYCX24_0608) and Interdisciplinary Innovation Fund for Doctoral Students of Nanjing University of Aeronautics and Astronautics (KXXCXJJ202505).

REFERENCES

- [1] Ursin Brunner and Kurt Stockinger. 2021. ValueNet: A Natural Language-to-SQL System that Learns from Database Information. In *ICDE*. 2177–2182.
- [2] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. 2024. Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. *Proc. VLDB Endow.* 17, 5 (2024), 1132–1145.
- [3] Jiaqi Guo, Zecheng Zhan, Yan Gao, and et al. 2019. Towards Complex Text-to-SQL in Cross-Domain Database with Intermediate Representation. In *ACL*. 4524–4535.
- [4] Ralf Hartmut Güting, Thomas Behr, and Christian Düntgen. 2010. SECONDO: A Platform for Moving Objects Database Research and for Publishing and Integrating Research Implementations. *IEEE Data Eng. Bull.* 33, 2 (2010), 56–63.
- [5] Chia-Hsuan Lee, Oleksandr Polozov, and Matthew Richardson. 2021. KaggleDBQA: Realistic Evaluation of Text-to-SQL Parsers. In *ACL/IJCNLP*. 2261–2273.
- [6] Fei Li and H. V. Jagadish. 2014. Constructing an Interactive Natural Language Interface for Relational Databases. *Proc. VLDB Endow.* 8, 1 (2014), 73–84.
- [7] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin Chen-Chuan Chang, Fei Huang, Reynold Cheng, and Yongbin Li. 2023. Can LLM Already Serve as A Database Interface? A Big Bench for Large-Scale Database Grounded Text-to-SQLs. In *NeurIPS*.
- [8] Mengyi Liu, Xieyang Wang, Jianqiu Xu, Hua Lu, and Yongxin Tong. 2025. NALSpatial: A Natural Language Interface for Spatial Databases. *IEEE Trans. Knowl. Data Eng.* 37, 4 (2025), 2056–2070.
- [9] Yi Liu, Xiangyu Liu, Xiangrong Zhu, and Wei Hu. 2024. Multi-Aspect Controllable Text Generation with Disentangled Counterfactual Augmentation. In *ACL*. 9231–9253.
- [10] Ana-Maria Popescu, Oren Etzioni, and Henry A. Kautz. 2003. Towards a theory of natural language interfaces to databases. In *JUI*. 149–157.
- [11] Jaydeep Sen, Chuan Lei, Abdul Quamar, and et al. 2020. ATHENA++: Natural Language Querying for Complex Nested SQL Queries. *Proc. VLDB Endow.* 13, 11 (2020), 2747–2759.
- [12] Yongxin Tong, Jieying She, Bolin Ding, Libin Wang, and Lei Chen. 2016. Online mobile Micro-Task Allocation in spatial crowdsourcing. In *ICDE*. 49–60.
- [13] Xieyang Wang, Mengyi Liu, Jianqiu Xu, and Hua Lu. 2023. NALMO: Transforming Queries in Natural Language for Moving Objects Databases. *GeoInformatica* 27, 3 (2023), 427–460.
- [14] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir R. Radev. 2018. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In *EMNLP*. 3911–3921.
- [15] John M. Zelle and Raymond J. Mooney. 1996. Learning to Parse Database Queries Using Inductive Logic Programming. In *AAAI*. 1050–1055.
- [16] Chao Zhang, Yuren Mao, Yijiang Fan, Yu Mi, Yunjun Gao, Lu Chen, Dongfang Lou, and Jinshu Lin. 2024. FinSQL: Model-Agnostic LLMs-based Text-to-SQL Framework for Financial Analysis. In *SIGMOD/PODS*. 93–105.
- [17] Yi Zhang, Jan Deriu, George Katsogiannis-Meimarakis, Catherine Kosten, Georgia Koutrika, and Kurt Stockinger. 2023. ScienceBenchmark: A Complex Real-World Benchmark for Evaluating Natural Language to SQL Systems. *Proc. VLDB Endow.* 17, 4 (2023), 685–698.
- [18] Victor Zhong, Caiming Xiong, and Richard Socher. 2017. Seq2SQL: Generating Structured Queries from Natural Language using Reinforcement Learning. *CoRR abs/1709.00103* (2017).

大数据集。相比之下，混合方法 NALMO+ 展示了显著的优势，即使在大量数据集的情况下，也能保持相对高效的查询，达到了 68.37 的 TES。

总之，这些研究结果强调了NALMOBench基准所带来的挑战，进一步证明了文本到可执行文件的研究仍处于早期阶段，在该领域还有很大的探索和发展潜力。

6.5 实验讨论

查询复杂性主要由查询的内在难度决定。对于简单的 TI 查询，仅包含时间约束、目标对象和关系信息。结构化语言相对固定，仅使用时间操作符。因此，自然语言理解阶段和结构化语言生成阶段都保持简单，使得所有评估的方法都能在 TA 和 TP 上取得最高分。相比之下，中等复杂度查询，包括 RA、NN、J 和 TS 查询，需要更丰富的实体属性（例如时间、地点、关系、对象标识符和最近邻数量）。这种丰富的语义范围显著增加了自然语言解析的难度。同时，结构化语言的语法必须覆盖扩展的操作符词汇，导致操作符和实体配对出现组合爆炸。大语言模型表现出三大主要不足。

以 TS 查询为具体例子。相似度阈值必须在每个数据集上通过经验进行校准，这需要依赖特定数据集的实验和手动调整。然而，LLM 无法推断出最佳阈值，并且经常生成虚假的操作符或默认使用很少使用但有效的操作符，这会降低查询性能。在困难级别上，CR 和 DT 查询通常涉及多个连续处理步骤或嵌套查询结构。由于实际执行需要迭代查询处理，现有方法往往无法有效应对，TA 和 TP 指标都保持在 50% 以下。

7 结论

在本文中，我们介绍了 NALMOBench，这是一种新颖的基准，用于评估将自然语言查询（NLQs）转换为 MODs 可执行语言的能力和质​​量。NALMOBench 不仅覆盖了合成运动物体数据集，还涵盖了真实世界的轨迹数据。我们进一步提出了一种基于大语言模型（LLM）的生成领域特定数据库语料库的方法。该方法包括常规的运动物体查询以及来自复杂真实世界场景的查询。为了解决从文本到可执行语言（text-to-EXE）的问题，我们将问题拆分为两个子问题：（i）自然语言理解和（ii）结构化语言生成，并为每个阶段提供了基线算法。

我们的实验表明，NALMOBench 对现有的文本到可执行程序（text-to-EXE）方法和大型语言模型（LLM）构成了显著挑战。我们认为，NALMOBench 为在 MOD 领域评估文本到可执行系统建立了一个严格的新标准。NALMOBench 为应对实际轨迹数据使用的固有复杂性以及推动优化后翻译可执行语言效率的研究奠定了基础。

8 感谢

本工作得到江苏省研究生科研与实践创新计划（KYCX24_0608）以及南京航空航天大学博士生跨学科创新基金（KXKCXJJ202505）的资助。

参考文献

[1] Ursin Brunner 和 Kurt Stockinger. 2021. ValueNet: 一个从数据库信息学习的自然语言到 SQL 系统. 载于 ICDE. 2177–2182. [2] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, 和 Jingren Zhou. 2024. 基于大型语言模型的 Text-to-SQL: 基准评估. Proc. VLDB Endow. 17, 5 (2024), 1132–1145. [3] Jiaqi Guo, Zecheng Zhan, Yan Gao, 等. 2019. 面向跨领域数据库的复杂 Text-to-SQL 中间表示方法. 载于 ACL. 4524–4535. [4] Ralf Hartmut Güting, Thomas Behr, 和 Christian Düntgen. 2010. SECONDO: 移动对象数据库研究的平台及研究实现的发布与整合. IEEE Data Eng. Bull. 33, 2 (2010), 56–63. [5] Chia-Hsuan Lee, Oleksandr Polozov, 和 Matthew Richardson. 2021. KaggleDBQA: Text-to-SQL 解析器的现实评估. 载于 ACL/IJCNLP. 2261–2273. [6] Fei Li 和 H. V. Jagadish. 2014. 构建一个交互式自然语言界面